15

# TensorFlow & Keras

The other big deep learning library — and its friendly face

Build powerful models with very little code

**AI/ML Engineer Learning Series · Deep Dive Edition**

| | |
|---|---|
| **TensorFlow** | A powerful deep learning library by Google |
| **Keras** | The easy, friendly API that sits on top of it |
| **You write** | Keras (simple); TensorFlow runs underneath (fast) |
| **Strength** | Great for production, mobile, and large-scale apps |
| **vs PyTorch** | Same goals; you already know the concepts |

# What's Inside

Here is everything covered in this guide, in order:

| # | Section | What you'll learn |
|---|---|---|
| 1 | **What is TensorFlow?** | The library and Google's role |
| 2 | **Keras — The Friendly Face** | Why you write Keras, not raw TF |
| 3 | **Tensors Again** | Same idea as PyTorch tensors |
| 4 | **Building a Model (Sequential)** | The easiest way |
| 5 | **Compile, Fit, Evaluate** | The three-step Keras workflow |
| 6 | **Reading Training History** | The numbers Keras gives you |
| 7 | **A Full Example** | Start to finish |
| 8 | **Callbacks** | Smart helpers during training |
| 9 | **TensorFlow vs PyTorch** | Which to pick, when |
| 10 | **Saving and Loading** | Keeping your model |
| 11 | **Common Mistakes** | What to avoid |
| ★ | **Cheatsheet** | Quick reference |

# 1. What is TensorFlow?

**TensorFlow** (■■■■■■■■■) is a powerful deep learning library made by Google. Just like PyTorch, it lets you build and train neural networks. The two are the biggest names in deep learning, and they do the same job — they just feel a little different to use.

The name comes from **tensors** (the number arrays you learned in PyTorch) that **flow** through the network. Same tensor idea, same neural network idea. If you understood the last two topics, you already know 90% of what's happening here.

TensorFlow has a reputation for being strong in **production** — putting models into real apps, on phones, and at huge scale. Google uses it across its products.

> ■ *Good news: you don't usually write raw TensorFlow. You write Keras, which is much simpler. That's the next section, and it's the friendly part.*

# 2. Keras — The Friendly Face

**Keras** (■■■■■) is a simple, beginner-friendly interface that sits on top of TensorFlow. You write easy Keras code, and TensorFlow does the heavy lifting underneath. Keras is famous for letting you build a working network in just a few lines.

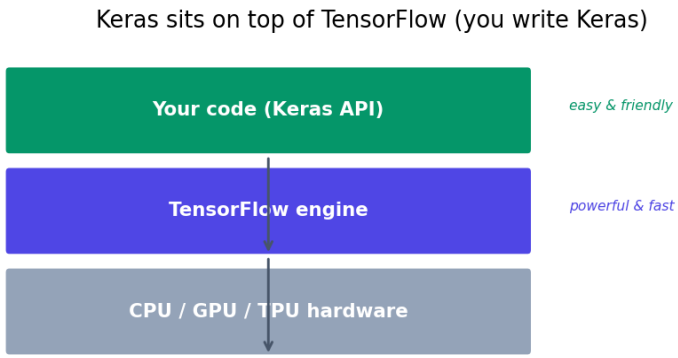


*Fig 1 — You write Keras (easy). TensorFlow runs underneath (powerful). Hardware does the math.*

Think of it like driving a car. Keras is the steering wheel and pedals — simple controls you actually touch. TensorFlow is the engine doing the real work. You don't need to understand every engine part to drive well.

> ■ *Since 2019, Keras is the official main way to use TensorFlow. So 'learning TensorFlow' for most people really means 'learning Keras'. That's what this guide focuses on.*

## 3. Tensors Again

TensorFlow tensors work just like PyTorch tensors — arrays of numbers that can run on a GPU. The commands look slightly different, but the idea is identical.

```
import tensorflow as tf

a = tf.constant([1, 2, 3])           # a vector
b = tf.constant([[1, 2], [3, 4]])    # a matrix

tf.zeros((2, 3))      # 2x3 of zeros
tf.ones((3, 3))       # 3x3 of ones
tf.random.normal((2, 2))   # random numbers

a.shape       # (3,)
a + a         # element-wise add
tf.matmul(b, b)   # matrix multiply

# Convert to / from NumPy
a.numpy()             # tensor -> numpy array
```

■ *Quick map: torch.tensor → tf.constant, torch.zeros → tf.zeros, A @ B → tf.matmul. Same concepts, different names. Don't overthink it.*

## 4. Building a Model — Sequential

The easiest way to build a network in Keras is **Sequential** — you just list the layers in order, top to bottom. It reads almost like plain English.

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(4,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(3, activation='softmax')   # 3 classes out
])

model.summary()   # prints a neat table of all layers
```

**Dense** is Keras's name for a normal fully-connected layer (PyTorch calls it Linear). Notice you write the activation right inside the layer — that's a small Keras convenience.

## 5. Compile, Fit, Evaluate — The Keras Workflow

This is where Keras shines. Where PyTorch makes you write a training loop by hand, Keras wraps it all into three simple commands:

| Step | Command | What it does |
|---|---|---|
| 1. Compile | model.compile(...) | Set the loss, optimizer, and metric |
| 2. Fit | model.fit(...) | Train the model (the whole loop, automatic) |
| 3. Evaluate | model.evaluate(...) | Check it on the test set |

```
# 1. Compile - set up how it learns
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

# 2. Fit - train it (this is the whole training loop!)
history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2)   # use 20% to check during training

# 3. Evaluate - test it
loss, acc = model.evaluate(X_test, y_test)
print('Test accuracy:', acc)

# Predict on new data
predictions = model.predict(X_new)
```

■ *That single model.fit() call replaces the entire 5-step PyTorch loop. This is why beginners often start with Keras — less code, fewer ways to make a mistake.*

## 6. Reading Training History

model.fit() returns a **history** object that records the loss and accuracy at every epoch — for both the training data and the validation data. Plotting it shows you how learning went.

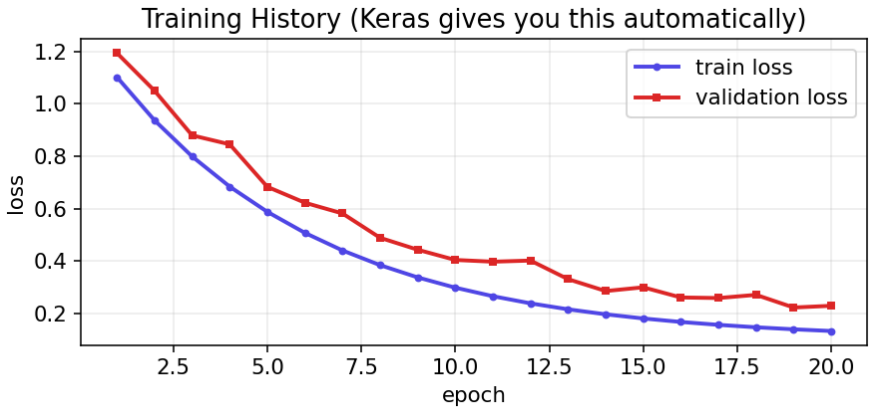


*Fig 2 — Training history. Both losses dropping together = good. A rising validation loss = overfitting.*

```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.xlabel('epoch'); plt.ylabel('loss'); plt.legend()
plt.show()
```

■ *Watch the gap between train and validation loss. If train loss keeps dropping but validation loss starts rising, the model is overfitting — time to stop or simplify.*

## 7. A Full Example, Start to Finish

Here is a complete Keras program on the iris flower dataset:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Data
X, y = load_iris(return_X_y=True)
X = StandardScaler().fit_transform(X)    # always scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. Model
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(4,)),
    layers.Dense(3, activation='softmax')
])

# 3. Compile, Fit, Evaluate
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=50, verbose=0)
loss, acc = model.evaluate(X_test, y_test)
print('Accuracy:', acc)
```

## 8. Callbacks — Smart Helpers

**Callbacks** (■■■■■■■) are little helpers that run during training to make it smarter. Two are extremely useful:

- **EarlyStopping** — stop training automatically when the model stops improving, so you don't waste time or overfit.
- **ModelCheckpoint** — save the best version of the model during training.

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

early_stop = EarlyStopping(
    monitor='val_loss',    # watch validation loss
    patience=5,            # wait 5 epochs before stopping
    restore_best_weights=True)

model.fit(X_train, y_train,
          epochs=100,
          validation_split=0.2,
          callbacks=[early_stop])
```

# 9. TensorFlow vs PyTorch

Both are excellent. They do the same things and reach similar results. Here's an honest comparison to help you choose.

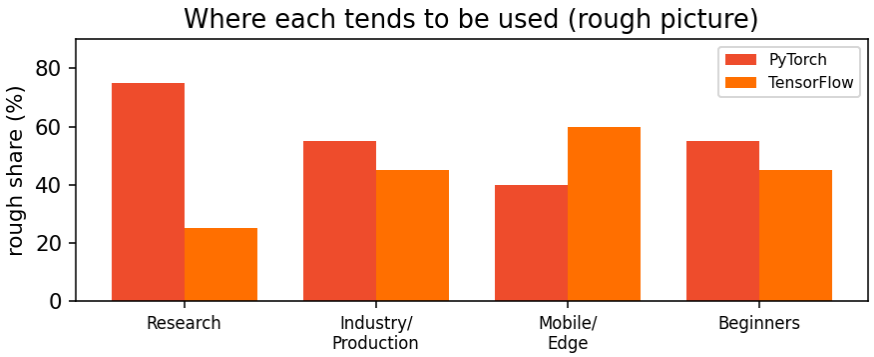


*Fig 3 — A rough picture of where each tends to be favoured.*

| | PyTorch | TensorFlow / Keras |
|---|---|---|
| Feel | Like normal Python | Keras is very beginner-friendly |
| Training loop | You write it (more control) | model.fit() does it for you |
| Research | Most popular choice | Also used, less dominant |
| Production / mobile | Good, improving | Very strong (TF Lite, TF Serving) |
| Learning curve | Gentle once you know the loop | Easiest to start with Keras |

**Honest advice:** learn PyTorch well (it's your main toolkit and dominates research), and know enough Keras to read and run TensorFlow code. Many jobs ask for one or the other, so being comfortable in both makes you flexible. The concepts transfer completely.

# 10. Saving and Loading

```
# Save the whole model (structure + weights)
model.save('my_model.keras')

# Load it back later
model = keras.models.load_model('my_model.keras')

# Use it right away
predictions = model.predict(X_new)
```

■ *Keras can save the entire model in one file — both the layout and the learned weights. In PyTorch you usually save just the weights (state_dict). Small difference, same goal.*

# 11. Common Mistakes

| Mistake | Why it's a problem | Fix |
|---|---|---|
| Not scaling inputs | Network trains poorly | Scale features first |
| Wrong loss name | Training fails or errors | Match loss to the task |
| Wrong final activation | Bad probabilities | softmax for multi-class output |
| Ignoring val_loss | Miss overfitting | Use validation_split + watch it |
| No EarlyStopping | Wastes time, overfits | Add the EarlyStopping callback |
| Mixing TF and PyTorch tensors | They don't mix | Pick one library per project |

# Quick Reference Cheatsheet

| Task | Code |
|---|---|
| Import | from tensorflow import keras |
| Make a tensor | tf.constant([1,2,3]) |
| Zeros / ones | tf.zeros((2,3)) / tf.ones((2,3)) |
| Build model | keras.Sequential([...]) |
| Dense layer | layers.Dense(16, activation='relu') |
| Output (multi-class) | layers.Dense(n, activation='softmax') |
| See the model | model.summary() |
| Compile | model.compile(optimizer='adam', loss=..., metrics=['accuracy']) |
| Train | model.fit(X, y, epochs=20, validation_split=0.2) |
| Evaluate | model.evaluate(X_test, y_test) |
| Predict | model.predict(X_new) |
| Early stopping | EarlyStopping(monitor='val_loss', patience=5) |
| Save model | model.save('model.keras') |
| Load model | keras.models.load_model('model.keras') |

**The Keras workflow to remember:** build (Sequential) → compile → fit → evaluate. Four steps, and you've trained a neural network.

**Next Topic:** CNN (Convolutional Neural Networks) — the special kind of neural network built for images. This is the engine behind your Product Image Classifier and Visual Defect Detector projects.